

```
--- djangoapi\scripts\crop\lineas_riego\lineas_riegoP00.txt ---
```

```
from psycpg.rows import dict_row
from myLib.gestor import tablasP00
```

```
class LineasRiegoP00(tablasP00):
```

```
    # INSERT
```

```
    def insert(self, d: dict):
        wkt = d['geom_wkt']
```

```
        if not self.geom_isValid(wkt):
            print("Error: La geometria no es valida.")
            self.disconnect()
            return None
```

```
        geom_final = self.snap_line_to_existing_network('lineas_riego', wkt)
```

```
        if not self.geom_isValid(geom_final):
            print("Error: La geometria ajustada no es valida.")
            self.disconnect()
            return None
```

```
        conflictos = self.get_interior_intersections('lineas_riego', geom_final)
        if conflictos:
```

```
            print(f"Error: La linea intersecta el interior de una existente. IDs en
conflicto: {conflictos}")
            self.disconnect()
            return None
```

```
        material = d['material']
        diametro_pulg = d['diametro_pulg']
        estado = d['estado']
        longitud_m = d['longitud_m']
```

```
        cons = """
            INSERT INTO lineas_riego
                (material, diametro_pulg, estado, longitud_m, geom)
            VALUES
                (%s, %s, %s, %s, ST_GeomFromText(%s, %s))
            RETURNING id
        """
```

```
        try:
```

```
            self.cur.execute(cons, [material, diametro_pulg, estado, longitud_m,
                                    geom_final, self.epsg])

            self.conn.commit()
            new_id = self.cur.fetchone()[0]
            print(f"Linea de riego insertada con id: {new_id}")
            return new_id
```

```
        except Exception as e:
```

```
            print(f"Error en insercion de linea de riego: {e}")
```

```

        self.conn.rollback()
        return None

    finally:
        self.disconnect()

# SELECTALL
def selectAll(self, d: dict):
    id_min = d['id_min']
    cons = """
        SELECT COUNT(*) AS total
        FROM lineas_riego
        WHERE id > %s
    """
    try:
        self.cur.execute(cons, [id_min])
        total = self.cur.fetchone()[0]
        print(f"{total} linea(s) de riego encontrada(s).")
        return total

    except Exception as e:
        print(f"Error en selectAll de lineas de riego: {e}")
        return 0

    finally:
        self.disconnect()

# SELECTASTUPLE
def selectAsTuple(self, d: dict):
    id_min = d['id_min']
    cons = """
        SELECT id, material, diametro_pulg, estado, longitud_m,
               ST_AsText(geom) AS geom_wkt
        FROM lineas_riego
        WHERE id > %s
    """
    try:
        self.cur.execute(cons, [id_min])
        rows = self.cur.fetchall()
        print(f"{len(rows)} linea(s) de riego encontrada(s).")
        return rows

    except Exception as e:
        print(f"Error en selectAsTuple de lineas de riego: {e}")
        return []

    finally:
        self.disconnect()

# SELECTASDICT
def selectAsDict(self, d: dict):
    id_min = d['id_min']
    cons = """

```

```

        SELECT id, material, diametro_pulg, estado, longitud_m,
               ST_AsText(geom) AS geom_wkt
        FROM lineas_riego
        WHERE id > %s
    """
    try:
        self.cur = self.conn.cursor(row_factory=dict_row)
        self.cur.execute(cons, [id_min])
        rows = self.cur.fetchall()
        print(f"{len(rows)} linea(s) de riego encontrada(s).")
        return rows

    except Exception as e:
        print(f"Error en selectAsDict de lineas de riego: {e}")
        return []

    finally:
        self.disconnect()

# UPDATE
def update(self, d: dict):
    wkt = d['geom_wkt']

    if not self.exists_by_id('lineas_riego', d['id']):
        print(f"Error: No existe la linea de riego con id {d['id']}.")
        self.disconnect()
        return None

    if not self.geom_isValid(wkt):
        print("Error: La geometria no es valida.")
        self.disconnect()
        return None

    geom_final = self.snap_line_to_existing_network('lineas_riego', wkt,
exclude_id=d['id'])

    if not self.geom_isValid(geom_final):
        print("Error: La geometria ajustada no es valida.")
        self.disconnect()
        return None

    conflictos = self.get_interior_intersections('lineas_riego', geom_final,
exclude_id=d['id'])
    if conflictos:
        print(f"Error: La linea intersecta el interior de una existente. IDs en
conflicto: {conflictos}")
        self.disconnect()
        return None

    cons = """
    UPDATE lineas_riego
    SET
        material      = %s,

```

```

        diametro_pulg = %s,
        estado         = %s,
        longitud_m      = %s,
        geom            = ST_GeomFromText(%s, %s)
    WHERE id = %s
"""
try:
    self.cur.execute(cons, [
        d['material'], d['diametro_pulg'], d['estado'], d['longitud_m'],
        geom_final, self.epsg, d['id']
    ])
    self.conn.commit()
    print(f"Linea de riego actualizada: {self.cur.rowcount} fila(s).")

except Exception as e:
    print(f"Error en update de lineas de riego: {e}")
    self.conn.rollback()

finally:
    self.disconnect()

# DELETE
def delete(self, d: dict):
    cons = "DELETE FROM lineas_riego WHERE id = %s"
    try:
        self.cur.execute(cons, [d['id']])
        self.conn.commit()
        print(f"Linea de riego eliminada: {self.cur.rowcount} fila(s).")

    except Exception as e:
        print(f"Error en delete de lineas de riego: {e}")
        self.conn.rollback()

    finally:
        self.disconnect()

```